



# Fix defects in floating-point `std::from_chars` (LWG3081, LWG3082, LWG3456)

**Document number:**

[P4168R0](#)

**Date:**

2026-04-05

**Audience:**

SG6

**Project:**

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

**Reply-to:**

Jan Schultke <[janschultke@gmail.com](mailto:janschultke@gmail.com)>

**GitHub Issue:**

[wg21.link/P4168/github](https://wg21.link/P4168/github)

**Source:**

[github.com/eisenwave/cpp-proposals/blob/master/src/fix-floating-from-chars.cow](https://github.com/eisenwave/cpp-proposals/blob/master/src/fix-floating-from-chars.cow)

---



**ABSTRACT:** This paper supersedes [\[P2827R1\]](#) and fixes [\[LWG3081\]](#), [\[LWG3082\]](#), and [\[LWG3456\]](#). It is intended as a defect report for C++17. The handling of floating-point overflow and underflow in `std::from_chars` is inconsistent; the implementations diverge from each other, and every implementation diverges from the wording in the standard.

## § Contents

- 1 **Introduction**
  - 1.1 Implementation divergence
  - 1.2 Defective wording
    - 1.2.1 Range of representable values
    - 1.2.2 No handling of non-ISO/IEC 60559 types
    - 1.2.3 No specification of parsing
    - 1.2.4 Signaling NaNs make `std::from_chars` unimplementable
- 2 **Design**



|     |                                   |
|-----|-----------------------------------|
| 2.1 | Limited design options            |
| 2.2 | Note on ISO/IEC 60559 conformance |
| 2.3 | Impact on existing code           |
| 2.4 | Alternatives considered           |
| 2.5 | Further edge cases                |
| 3   | Implementation experience         |
| 4   | Editorial problems                |
| 5   | Wording                           |
| 5.1 | [version.syn]                     |
| 5.2 | [charconv.to.chars]               |
| 5.3 | [charconv.from.chars]             |
| 6   | References                        |

## § 1. Introduction

In 2018, [LWG3081] pointed out that the `from_chars` API does not give the user a means of distinguishing floating-point overflow and underflow. The perceived defect is a loss of functionality compared to `std::strtod`. This issue remains open to this day despite having Priority 2.



**EXAMPLE:** An example of code affected by this defect looks as follows:

```
const char str[] = "1e100000";  
float x; // assuming float in binary32 ISO/IEC 60559 format  
const auto [p, ec] = std::from_chars(std::begin(str), std::end(str), x);
```

libstdc++ leaves `x` unmodified, but libc++ and MSVC STL set it to  $\infty$ . `ec` is `std::errc::result_out_of_range` for all implementations, but the wording requires that `x` is set to  $\infty$  and that `ec` is value-initialized, which no one implements.

In 2023, [P2827R1] attempted to solve this issue, but died in LEWG eventually. Lack of motivation within the paper was among the feedback given. Perplexingly, [P2827R1] does not mention the implementation divergence between libstdc++ and MSVC STL, which already existed at the time; mentioning it surely would have added motivation.

The issue is much more severe than the LWG issue and paper make it out to be. The wording is defective, implementations diverge, and users suffer from it to the point where `boost::from_chars_erange` ([BoostCharConv]) should arguably be recommended to users over the standard feature; at least it is portable and well-specified.

### § 1.1. Implementation divergence

There are three cases of interest where typical implementations likely diverge:



- Treatment of floating-point overflow (always resulting in infinity), such as when parsing `"1e+1000"` as `std::float32_t`.
- Treatment of floating-point underflow that results in an inexact zero, such as when parsing `"1e-10000"` as `std::float32_t`.
- Treatment of floating-point underflow that results in an inexact subnormal, such as when parsing `"1e-45"` as `std::float32_t`.



**NOTE:** ISO/IEC 60559 leaves the definition of “tininess” up to the implementation. This means that it is unclear whether the `"1e-45"` case is analogous to floating-point underflow.

At the time of writing, implementations behave as shown in the table below when parsing a string as `float` (binary32). Each cell shows what value the given `float` is set to, as well as the returned `std::errc` value.

| Input                    | libstdc++                           | libc++                                     | MSVC STL                                   |
|--------------------------|-------------------------------------|--------------------------------------------|--------------------------------------------|
| <code>"0"</code>         | <code>0.f / {}</code>               | <code>0.f / {}</code>                      | <code>0.f / {}</code>                      |
| <code>"#"</code>         | unmodified /<br>invalid_argument    | unmodified /<br>invalid_argument           | unmodified /<br>invalid_argument           |
| <code>"1e-45"</code>     | <code>1e-45f / {}</code>            | <code>1e-45f / {}</code>                   | <code>1e-45f / {}</code>                   |
| <code>"1e-10000"</code>  | unmodified /<br>result_out_of_range | <code>0.f /</code><br>result_out_of_range  | <code>0.f /</code><br>result_out_of_range  |
| <code>"-1e-10000"</code> | unmodified /<br>result_out_of_range | <code>-0.f /</code><br>result_out_of_range | <code>-0.f /</code><br>result_out_of_range |
| <code>"1e+10000"</code>  | unmodified /<br>result_out_of_range | <code>+∞ /</code><br>result_out_of_range   | <code>+∞ /</code><br>result_out_of_range   |
| <code>"-1e+10000"</code> | unmodified /<br>result_out_of_range | <code>-∞ /</code><br>result_out_of_range   | <code>-∞ /</code><br>result_out_of_range   |



**NOTE:** The libc++ and MSVC STL behavior is proposed.

## § 1.2. Defective wording

### § 1.2.1. Range of representable values

Out of the implementations above, none comply with the existing wording. `[charconv.from.chars]` states:



[...] If the parsed value is not in the **range representable by the type** of value, value is unmodified and the member `ec` of the return value is equal to `errc::result_out_of_range`. Otherwise, value is set to the parsed value, after rounding according to `round_to_nearest ([round.style])`, and the member `ec` is value-initialized.

The problem is that for several years while `std::from_chars` existed, there was no definition of “range of representable values” (marked above) in the standard. That definition was only added recently in 2023 via [CWG2723]; the definition in [basic.fundamental] paragraph 13 includes *all real numbers* in that range, meaning that every standard library is incorrect for returning `errc::result_out_of_range` on overflow and underflow. Ergo, not only is underflow indistinguishable from overflow via error code, overflow, underflow, and exact rounding are all indistinguishable if the wording is implemented (which it isn't).

### § 1.2.2. No handling of non-ISO/IEC 60559 types

Another problem with the wording is that it does not account for types that do not have a representation of infinity or NaN. The string pattern accepted by `std::from_chars` is always permitted to be `"INFINITY"`, but what should happen when returning a floating-point type without infinity representations?

### § 1.2.3. No specification of parsing

Last but most severe, the entire mechanism of mapping the matched pattern onto a floating-point value is described as:

” Otherwise, value is set to the parsed value, after rounding according to `round_to_nearest` ([round.style]), and the member `ec` is value-initialized.

The fact that a string such as `"INF"` results in infinity or that `"1"` results in `1.0` is handwaved as the single word “parsed”, which is not explained anywhere. While most such cases are obvious, `std::to_chars` in `libc++` represents signaling NaNs as `"nan(snan)"`, and it is unclear whether `std::from_chars` can recover a signaling NaN from such a string because there exists no wording for “parsed”.

The intent is presumably to handle these cases like `std::strtod`, but this is not stated anywhere; only the syntax of the matched pattern is specified, not the interpretation of the pattern. The description of the matched pattern is also deemed underspecified, as explained in [LWG3456].

### § 1.2.4. Signaling NaNs make `std::from_chars` unimplementable

Yet another problem is that `std::from_chars` cannot be implemented for types that can represent signaling NaNs; the problem lies in [charconv.to.chars] paragraph 2:

” The functions that take a floating-point value but not a precision parameter ensure that the string representation consists of the smallest number of characters such that there is at least one digit before the radix point (if present) and parsing the representation using the corresponding `from_chars` function recovers value exactly.

It is not entirely clear whether this requirement was intended to apply to non-finite inputs, but it technically could be read in such a way. If so, it would additionally mandate that `"inf"` instead of `"infinity"` is emitted when the value is  $+\infty$ , which implementations do.

The problem is that the pattern accepted by `std::from_chars` is that of `std::strtod`, which supports `"INFINITY"`, `"NaN"`, and `"NaN(d-char-sequence)"` but no `"SNAN"` as inputs. `std::strtod` interprets every NaN input as a quiet NaN, which is presumably the intent for `std::from_chars`, even if the wording does not explicitly say so; in fact, the wording does not say anything about how these text representations of infinity or NaN are interpreted as a floating-point value by `std::from_chars`.

As things stand, the requirement to “recover the value exactly” cannot be implemented for signaling NaNs in any reasonable way. Therefore, it is necessary to exempt signaling NaNs from being recovered. `std::strtod` has this exemption, as does ISO/IEC 60559; see §5.12.1 “External character sequences representing zeros, infinities, and NaNs”:

” Conversion of a signaling NaN in a supported format to an external character sequence should produce a language-defined one of `"snan"` or `"nan"` or a sequence that is equivalent except for case, with an optional preceding sign.

## § 2. Design

In short, the MSVC STL and libc++ behavior is proposed, with additional wording clarifications.

### § 2.1. Limited design options

At this stage, the available design options are extremely limited because floating-point implementations of `std::from_chars` have already existed for years (MSVC STL since 2018, libstdc++ since 2021, libc++ since 2025). Changing the behavior of `std::from_chars` can break substantial amounts of existing code.

There seem to be only two plausible options with sufficiently low impact:

- Standardize the libstdc++ behavior, which is to leave the given `float&` unmodified on overflow/underflow.
- Standardize the MSVC STL and libc++ behavior, which is to overwrite the given `float&` with either zero or infinity.

I argue that the libc++ behavior is strictly better because

- the behavior resolves [\[LWG3081\]](#) by making underflow distinguishable from overflow,
- users may actually want to ignore `result_out_of_range` and simply take the 0 resulting from underflow and  $\infty$  resulting from overflow, which the libc++ behavior makes possible, and
- the behavior is consistent with `std::strtod`.

To provide the maximum amount of information to the user, it is also important that the user obtains a correctly signed zero on underflow and a correctly signed  $\infty$  on overflow. That is, the sign of the result value should always be the sign of the parsed mathematical value, even if rounded to zero or infinity. Both MSVC STL and libc++ provide correctly signed values.

## § 2.2. Note on ISO/IEC 60559 conformance



The choice of (correctly signed) zeros and infinities to signal underflow and overflow is far from arbitrary; `strtod` has that design, and correctly implements the ISO/IEC 60559 (or IEEE-754) operation “convertFromDecimalCharacter” for floating-point types (see C23 §F.3).

`std::from_chars` should have that same behavior to increase ISO/IEC 60559 conformance. `[charconv.from_chars]` even uses the `round_to_nearest` rounding mode corresponding to “roundTiesToEven” already, which makes it extremely similar to the ISO/IEC 60559 operation.

As mentioned in §1.2.4. Signaling NaNs make `std::from_chars` unimplementable, it is also necessary to handle signaling NaNs in some way, as part of the “convertFromDecimalCharacter” operation. The easiest way to do so would be to double down on the practice of always producing “NAN” for both quiet and signaling NaNs; the wording merely needs to relax the exact recoverability requirement for that case.

## § 2.3. Impact on existing code

Furthermore, we must consider how the change in behavior affects existing users, if we standardize the behavior of the other implementation:

- Users of `libstdc++` likely ignore the written value on `result_out_of_range` because it is unmodified. Now modifying the value does not cause any problem if they were ignoring it already.
- Users of MSVC STL and `libc++` may rely on infinity or zero being written on `result_out_of_range`, and suddenly leaving the value unmodified would result in use of erroneous or even indeterminate values for these users.

When considering these two options, standardizing the GCC behavior appears more risky.

## § 2.4. Alternatives considered

Neither [LWG3081] nor [P2827R1] fully resolve the defects in the wording. Each proposes a different value that should be written for overflow; the LWG issue proposes `numeric_limits<T>::max()`, and the paper proposes 1.

Neither of these behaviors standardizes existing practice in standard libraries, is particularly well-motivated, or matches the behavior of other functions such as `std::strtod`. Both alternatives make it difficult to simply ignore the `result_out_of_range` error and use the value.

## § 2.5. Further edge cases

As explained in §1.2.2. No handling of non-ISO/IEC 60559 types, there are various edge cases such as parsing the string “INFINITY” and attempting to store it in a floating-point type that does not have a representation of infinity. These edge cases are currently not handled in any explicit way by major standard library implementations. To my knowledge, all major standard

libraries assume ISO/IEC-60559-like types, meaning that negative and positive infinity and NaN are also supported.



The proposed solution is to handle these cases as closely as possible to `std::strtod`, as specified in [C23 §7.24.1.5](#). For example, this means that when `"inf"` is parsed but not representable in the type, the greatest finite value is returned instead, and the error code is `errc::result_out_of_range`.

## § 3. Implementation experience

The proposed behavior has been released in MSVC STL and libc++, except that the behavior in [§2.5. Further edge cases](#) does not need to be implemented and only exists on paper for these implementations.

The proposed behavior has also been implemented as `boost::from_chars_erange` ([\[BoostCharConv\]](#)).

## § 4. Editorial problems

Beyond the normative issues, there are also significant editorial problems with [\[charconv.from.chars\]](#) which make it difficult to understand what pattern it actually accepts. Namely, the wording is currently:

”

*Effects:* The pattern is the expected form of the subject sequence in the "C" locale, as described for `strtod`, except that

- the sign '+' may only appear in the exponent part;
- if `fmt` has `chars_format::scientific` set but not `chars_format::fixed`, the otherwise optional exponent part shall appear;
- if `fmt` has `chars_format::fixed` set but not `chars_format::scientific`, the optional exponent part shall not appear; and
- if `fmt` is `chars_format::hex`, the prefix `"0x"` or `"0X"` is assumed.

In any case, the resulting value is one of at most two floating-point values closest to the value of the string matching the pattern.

We don't actually get to understand what the pattern is here, just that we should look at `strtod` and make mental changes relative to that. [C23 §7.24.1.5 paragraph 3](#) then describes the sequence partially using a mixture of prose and grammar, containing descriptions such as

”

[...] a nonempty sequence of decimal digits optionally containing a decimal-point character, then an optional exponent part as defined in 6.4.4.3, excluding any digit separators (6.4.4.2);

Even the C wording doesn't directly describe the pattern but refers to various other definitions such as “exponent part” and “digit separators” that must be looked up elsewhere. Anyone trying to understand what `std::from_chars` actually accepts needs to go through a multi-standard scavenger hunt and keep track of two layers of “excluding” and “except that”.

This is an ineffective way to communicate the behavior of `std::from_chars` and should be rewritten so that the accepted pattern can be understood solely through [\[charconv.from.chars\]](#). [\[LWG3456\]](#) performs such a rewrite, but does not fully decouple `std::from_chars` from the C wording.

A proper rewrite can also fix the lack of clarity pointed out in [\[LWG3082\]](#).

## § 5. Wording

The changes are relative to [\[N5032\]](#).

### § [version.syn]

Bump the feature-test macro in [\[version.syn\]](#) as follows:



```
#define __cpp_lib_to_chars 202306L 20XXXXL // freestanding, also in <charconv>
```



**DRAFTING NOTE:** There is no separate macro for `std::from_chars`. `__cpp_lib_to_chars` can be viewed as a representative feature-test for all of `<charconv>`.

### § [charconv.to.chars]

Change [\[charconv.to.chars\]](#) paragraph 2 as follows:



The functions that take a floating-point value but not a precision parameter ensure that when the given value is finite, the string representation consists of the smallest number of characters such that there is at least one digit before the radix point (if present) and parsing the representation using the corresponding `from_chars` function recovers value exactly.

[Note: This guarantee applies only if `to_chars` and `from_chars` are executed on the same implementation. — end note]

If there are several such representations, the representation with the smallest difference from the floating-point argument value is chosen, resolving any remaining ties using rounding according to `round_to_nearest` ([\[round.style\]](#)).

### § [charconv.from.chars]



¶ All functions named `from_chars` analyze the string `[first, last)` for a pattern, where `[first, last)` is required to be a valid range. If no characters match the pattern, `value` is unmodified, the member `ptr` of the return value is `first` and the member `ec` is equal to `errc::invalid_argument`.

[Note: If the pattern allows for an optional sign, but the string has no digit characters following the sign, no characters match the pattern. — *end note*]

Otherwise, the characters matching the pattern are interpreted as a representation of a value of the type of `value`. The member `ptr` of the return value points to the first character not matching the pattern, or has the value `last` if all characters match. ~~If the parsed value is not in the range representable by the type of `value`, `value` is unmodified and the member `ec` of the return value is equal to `errc::result_out_of_range`. Otherwise, `value` is set to the parsed value, after rounding according to `round_to_nearest` (`[round.style]`), and the member `ec` is `value-initialized`.~~

```
constexpr from_chars_result from_chars(const char* first, const char* last,
                                       integer_type& value, int base = 10)
```



¶ Let the pattern be the maximal sequence of characters starting with `*first` that matches a *from-chars-signed-integer-pattern* if `value` is of a signed type, and matches a *from-chars-unsigned-integer-pattern* otherwise.

*from-chars-signed-integer-pattern:*  
`-opt from-chars-unsigned-integer-pattern`  
*from-chars-unsigned-integer-pattern:*  
`base-digit-seq`  
*base-digit-seq:*  
`base-digit base-digit-seqopt`

*base-digit* is a digit character whose value is less than `base`. The code points U+0030..U+0039 DIGIT ZERO..NINE represent digit characters with value 0..9, respectively; both U+0041..U+005A LATIN CAPITAL LETTER A..Z and U+0061..U+007A LATIN SMALL LETTER A..Z represent digit characters with value 10..35, respectively.



**DRAFTING NOTE:** The description deliberately uses code points to avoid ever spelling a character literal, in preparation for [P3876R1], which adds support for `char8_t` and other character types.

¶ *Preconditions:* `base` has a value between 2 and 36 (inclusive).



¶ *Effects:* The pattern is the expected form of the subject sequence in the "C" locale for the given nonzero `base`, as described for `strtol`, except that

no "0b" or "0B" prefix shall appear if the value of base is 2, no "0x" or "0X" prefix shall appear if the value of base is 16, and except that '-' is the only sign that may appear, and only if value has a signed type.



¶ *Effects*: If the pattern is empty, the effect is as described above. Otherwise, an integer value  $x$  is obtained by interpreting the *base-digits* as digits of an integer in the given base, and negating that value if the leading U+002D HYPHEN MINUS is present. If  $x$  is in the range of values representable by the type of value, value is set to  $x$  and the member ec of the result is value-initialized; otherwise, value is unmodified and the member ec of the result is `errc::result_out_of_range`.

¶ *Throws*: Nothing.

```
from_chars_result from_chars(const char* first, const char* last,
                             floating-point-type& value,
                             chars_format fmt = chars_format::general);
```



¶ Let the pattern be the maximal sequence of characters starting with \*first that matches a *from-chars-floating-pattern*.

*from-chars-floating-pattern*:

*-<sub>opt</sub> from-chars-unsigned-floating-pattern*

*from-chars-unsigned-floating-pattern*:

*from-chars-fixed-float*

*from-chars-scientific-float*

*from-chars-general-float*

*from-chars-hex-float*

*from-chars-infinity*

*from-chars-nan*

*from-chars-fixed-float*:

*digit-sequence decimal-fraction<sub>opt</sub>*

*from-chars-scientific-float*:

*digit-sequence decimal-fraction<sub>opt</sub> exponent-part*

*from-chars-general-float*:

*digit-sequence decimal-fraction<sub>opt</sub> exponent-part<sub>opt</sub>*

*from-chars-hex-float*:

*hexadecimal-digit-sequence hexadecimal-fraction<sub>opt</sub> binary-exponent-part<sub>opt</sub>*

*decimal-fraction*:

*. digit-sequence*

*hexadecimal-fraction*:

*. hexadecimal-digit-sequence*

*from-chars-infinity*:

*INF*

*INFINITY*



```

from-chars-nan:
    NAN
    NAN( nan-char-seqopt )

nan-char-seq:
    nan-char nan-char-seqopt

nan-char:
    digit
    nondigit

```

¶ Matching of a *from-chars-floating-pattern* is performed so that

- *from-chars-fixed-float*, *from-chars-scientific-float*, *from-chars-general-float*, and *from-chars-hex-float* may only be matched if `fmt` equals `chars_format::fixed`, `chars_format::scientific`, `chars_format::general`, and `chars_format::hex`, respectively;
- case is ignored in *from-chars-infinity* and in the leading NAN part of a *from-chars-nan*; and
- a “'” digit separator is not matched anywhere in a *from-chars-floating-pattern*.

¶ *Preconditions*: `fmt` has the value of one of the enumerators of `chars_format`.



¶ *Effects*: The pattern is the expected form of the subject sequence in the “C” locale, as described for `strtod`, except that

- the sign ‘+’ may only appear in the exponent part;
- if `fmt` has `chars_format::scientific` set but not `chars_format::fixed`, the otherwise optional exponent part shall appear;
- if `fmt` has `chars_format::fixed` set but not `chars_format::scientific`, the optional exponent part shall not appear; and
- if `fmt` is `chars_format::hex`, the prefix “0x” or “0X” is assumed.

In any case, the resulting value is one of at most two floating-point values closest to the value of the string matching the pattern.



¶ *Effects*:

- If the pattern is empty, the effect is as described above.
- Otherwise, if the pattern is of the form “– *from-chars-infinity*”,
  - if negative infinity is representable in the type of `value`, `value` is set to negative infinity and the member `ec` of the result is value-initialized;
  - otherwise, `value` is set to the lowest finite value representable in the type of `value` and the member `ec` of the result is



`errc::result_out_of_range`.

- Otherwise, if the pattern is of the form *from-chars-infinity*,
  - if either positive or unsigned infinity is representable in the type of `value`, `value` is set to such an infinity and the member `ec` of the result is `value-initialized`;
  - otherwise, `value` is set to the greatest finite value representable in the type of `value` and the member `ec` of the result is `errc::result_out_of_range`.
- Otherwise, if the pattern is of the form “*-<sub>opt</sub>from-chars-nan*”,
  - if quiet NaN is representable in the type of `value`, `value` is set to quiet NaN and the member `ec` of the result is `value-initialized`, where the meaning of the optional *nan-char-seq* is implementation-defined,
  - otherwise, `value` is set to positive or unsigned zero and the member `ec` of the result is `errc::invalid_argument`.
- Otherwise, the character sequence represents a rational number  $q$  whose value is determined by
  - interpreting the pattern without leading U+002D HYPHEN MINUS but with an added *hexadecimal-prefix* as a *hexadecimal-floating-point-literal* if `fmt` is `chars_format::hex`, and
  - interpreting the pattern without leading U+002D HYPHEN MINUS as a *decimal-floating-point-literal* otherwise,

then interpreting the scaled value (`[lex.fcon]`) of that literal as the value of  $q$ , negated if the leading U+002D HYPHEN MINUS is present.  $q$  is correctly rounded to a value  $r$  according to `round_to_nearest` (`[round.style]`). `value` is set to  $r$ , and the member `ec` of the result is

- `value-initialized` if `isnormal( $r$ )` is `true`,
- `errc::result_out_of_range` if  $r$  is negative or positive infinity or  $q$  is not in the range of values representable by the type of `value`,
- `errc::result_out_of_range` if  $r$  is zero but  $q$  is not zero, or
- an implementation-defined choice of either `errc::result_out_of_range` or a `value-initialized` result if  $r$  is a subnormal.

¶ Throws: Nothing.

See also: ISO/IEC 9899:2024, 7.24.2.6, 7.24.2.8

## § 6. References



[N5032] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-12-15)  
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf>

[P2827R1] *Zhihao Yan*. Floating-point overflow and underflow in `from_chars` (LWG 3081) (2023-11-20)  
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2827r1.html>

[P3876R1] *Jan Schultke, Peter Bindels*. Extending `<charconv>` support to more character types  
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3876r1.html>

[LWG3081] *Jiang An*. Floating point `from_chars` API does not distinguish between overflow and underflow (2018-03-12)  
<https://wg21.link/LWG3081>

[LWG3082] *Greg Falcon*. `from_chars` specification regarding floating point rounding is inconsistent (2018-03-12)  
<https://wg21.link/LWG3082>

[LWG3456] *Jonathan Wakely*. Pattern used by `std::from_chars` is underspecified (2020-06-23)  
<https://wg21.link/LWG3456>

[CWG2723] *Jiang An*. Range of representable values for floating-point types (2023-04-21)  
<https://cplusplus.github.io/CWG/issues/2723.html>

[BoostCharConv] *CharConv: An Implementation of `<charconv>` in C++11*  
<https://www.boost.org/doc/libs/latest/libs/charconv/doc/html/charconv.html>